

Preferred message sequences for communicating with RATS (Registration And Timing System)

1	Connecting timing gates to RATS.....	2
2	Message format.....	2
3	Preferred message usage	2
3.1	Background	2
3.2	Precedence	2
3.3	Recognised timing gate states	3
4	Messages sent from the RATS PC to the timing gates	4
5	Messages sent from the timing gates to RATS	5
6	Example 3 Gate Maze State machine for timing actions.....	6
6.1	3 Gate State Diagram	6
6.2	3 Gate State Actions	6
7	Example 2 Gate Line Follower State machine for timing actions	8
7.1	2 Gate State Diagram	8
7.2	2 Gate State Actions	8
8	Example 1 Gate Line Follower State machine for timing actions	10
8.1	1 Gate State Diagram	10
8.2	1 Gate State Actions	10
9	Annex A - All RATS message types.....	12

Preferred message sequences for communicating with RATS (Registration And Timing System)

1 Connecting timing gates to RATS

RATS accepts messages from timing gates through a USB serial connection with these characteristics:

- BaudRate = 9600
- DataBits = 8
- Parity = Parity.None
- StopBits = StopBits.One

2 Message format

All RATS messages are in the format "<message_type,value>CrLf" where:

- message_type is an integer representing the action of the message
- value is an integer value the value relating to the message

3 Preferred message usage

3.1 Background

RATS can work with timing gates that have 1,2 or 3 gates and examples of how each of these configurations can interact with RATS are included in this document.

RATS has evolved over several years and is capable of working in multiple modes, the description here is of the preferred method of communicating and does not describe all the legacy capabilities included in RATS for backward compatibility with older timing gates.

3.2 Precedence

Values from the attached timing gates always take precedence over interpolated values used by RATS to maintain running displays. The run time recorded for a robot run is always that provided by the gates to maximise accuracy of timing. Similarly, if the timing gates supply an updated course time (time a robot has used from the allowed entry time) then that will override any internally calculated value in RATS.

Preferred message sequences for communicating with RATS (Registration And Timing System)

3.3 Recognised timing gate states

RATS recognises the following timing gate states:

State No	Name	Manadatory	Comments
0	Calibrating		
1	Waiting for robot in start position		
2	Robot in start position	Yes	Reporting this state will cause RATS to clear the split time and reset the number of touches for a run
3	Started		
4	Running	Yes	Reporting this state will cause RATS to increment the number of runs started by a robot and check this against the number of runs allowed
5	Finishing		

Preferred message sequences for communicating with RATS (Registration And Timing System)

4 Messages sent from the RATS PC to the timing gates

The following messages are used to send information from RATS:

message_type	Name	Description	Usage
98	<i>NewMouse</i>	Start of an entry for a new mouse.	Tells the timing gates to reset themselves to a state ready for a new robot to commence its entry.
96	<i>ContestName</i>	Name of current contest eg "Senior Maze Solver"	Information passed to timing gates. It is up to the timing gate system how, or if, these information messages are handled.
95	<i>EventName</i>	Name of current robotics event eg "Minos"	Information passed to timing gates. It is up to the timing gate system how, or if, these information messages are handled.
94	<i>AllowedRuns</i>	Number of runs allowed for this entry	Information passed to timing gates. It is up to the timing gate system how, or if, these information messages are handled.
93	<i>EntryTimeS</i>	Time allowed in seconds for this entry to complete runs	Information passed to timing gates. It is up to the timing gate system how, or if, these information messages are handled.
92	<i>ExtraRun</i>	Allow an extra run by decrementing the run count	RATS requests the Gate timer to decrement the current count of runs to allow an extra run for the current entry. This optional, timing gates are not required to keep a count of runs for entries.

Preferred message sequences for communicating with RATS (Registration And Timing System)

5 Messages sent from the timing gates to RATS

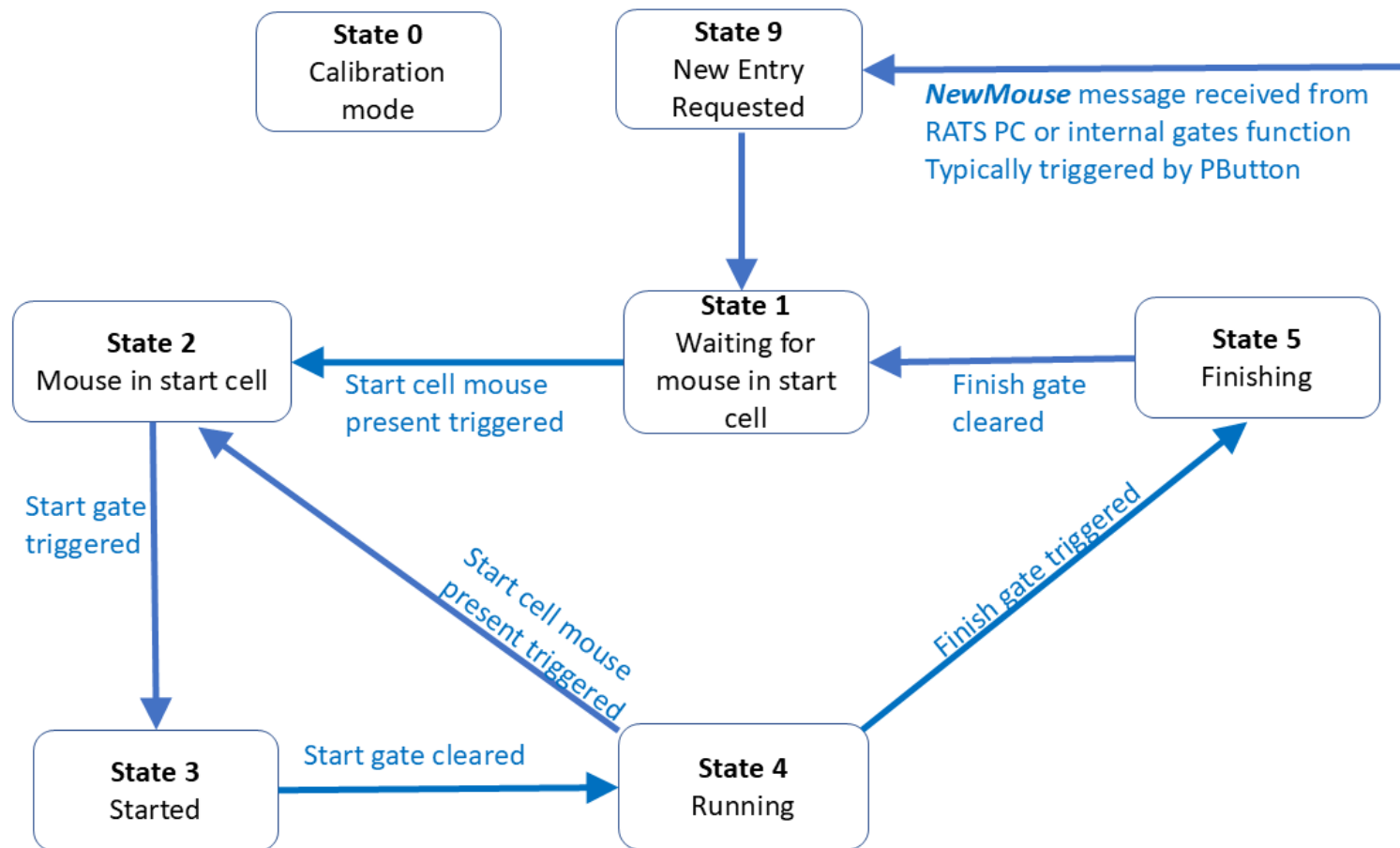
The following messages are used to send information to RATS:

message_type	Name	Description	Usage
30	<i>CourseTimeMs</i>	Length of time in milliseconds that the current mouse has been active in the maze.	Tells RATS to start timing the time used by a robot entry and implement the entry time constraints for the contest. Must be sent with the value 0 at the start of a robot entry. It is not necessary to send updates after this, but if they are sent, they will override the values calculated in RATS.
12	<i>C1SplitTime</i>	Channel 1: Length of time in milliseconds for the current mouse on its current run.	Tells RATS to start reporting the timing of a robot run. The split time is for information only whilst a robot is running. This value is not recorded as a run time for a mouse. Must be sent with the value 0 when the robot triggers the start gate. Not necessary to send updates after this, but if they are sent, they will override the values calculated in RATS.
13	<i>C1RunTime</i>	Channel 1: Length of time in milliseconds for a run that has just completed.	Tells RATS the definitive run time to be stored and used to calculate a score time. The value must be sent when the robot triggers the finish gate. By convention this message is repeated for a total of 3 transmissions to mitigate any line errors preventing a valid run time being recorded. RATS will only record the run time once and discards additional messages for the same run time.
4	<i>TimerState</i>	Timer state reported from gates to RATS.	Reports the state number of the timing gates. States 2 and 4 must be reported. Reporting other states is optional.
0	<i>WatchDog</i>	Resets the watch dog timer in RATS.	Timing gates should send a <i>WatchDog</i> message with value 0 at least every second. RATS expects to receive a <i>WatchDog</i> message from the timing gates to confirm that the gates are running and communication is working. If more than 2 seconds elapses between <i>WatchDog</i> messages, RATS reports an error to alert judges to problems with the timing system. For older legacy timing gates RATS has the capability to suppress the watchdog error checking.
91	<i>RequestTouch</i>	Requests RATS to add a touch penalty.	Requests RATS to apply a touch penalty to the current run. This is optional for gate controllers. The RATS control console can apply touch penalties directly.

Preferred message sequences for communicating with RATS (Registration And Timing System)

6 Example 3 Gate Maze State machine for timing actions

6.1 3 Gate State Diagram



6.2 3 Gate State Actions

State	Name	Predecessors	Entry condition	Actions	Successors	Exit Condition
-------	------	--------------	-----------------	---------	------------	----------------

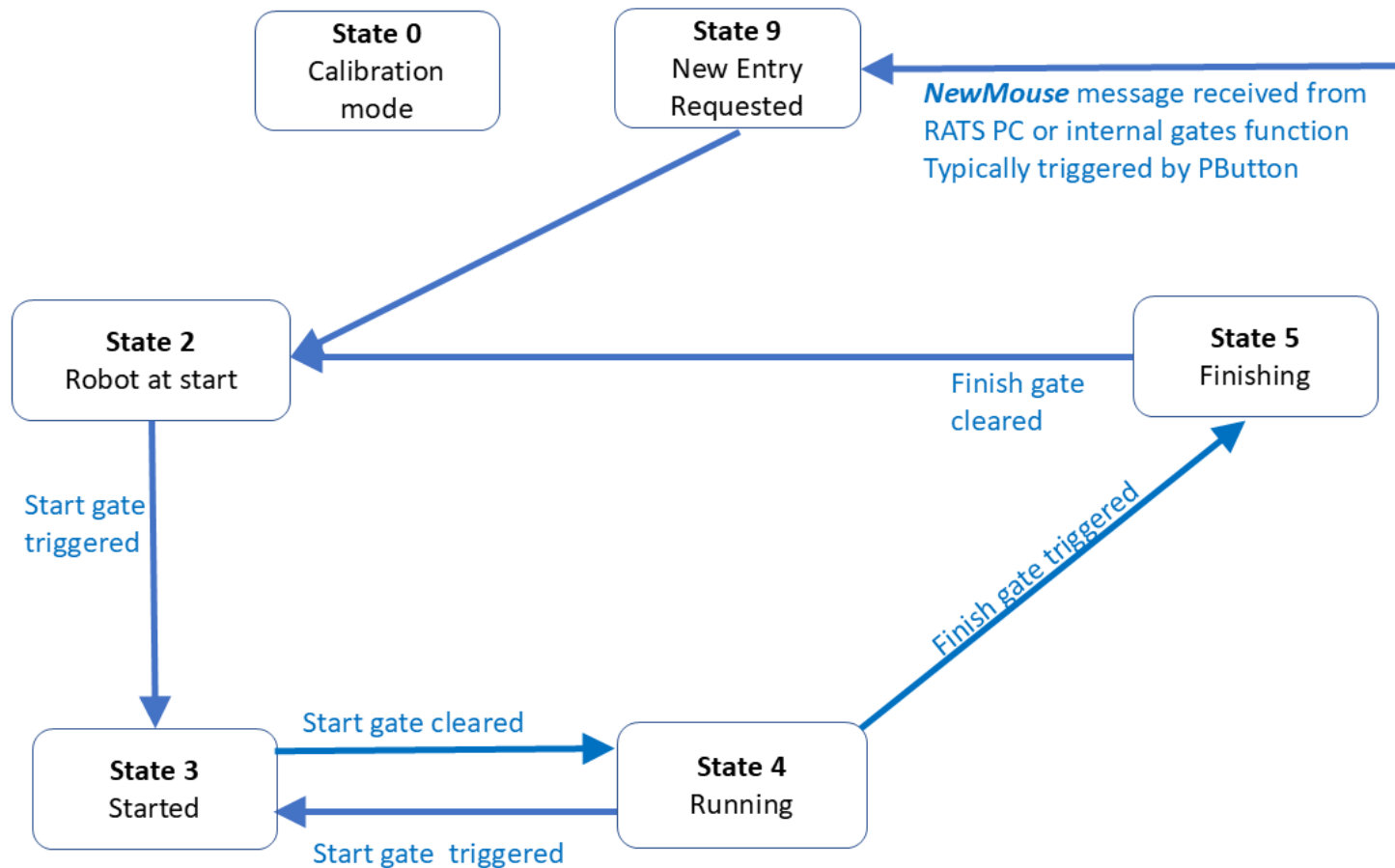
Preferred message sequences for communicating with RATS (Registration And Timing System)

0	Calibrating	Initial state	Power On	Wait NewMouse from RATS	9 New Entry	NewMouse received
1	Waiting Robot	9 New Entry 5 Finishing	Immediate Finish gate cleared	Send < TimerState,1 > to RATS Wait for robot present at start to be triggered	2 Robot at Start	Robot at start triggered
2	Robot at Start	1 Waiting Robot 4 Running	Robot at start triggered	Send < TimerState,2 > to RATS Wait for start gate to be triggered	3 Started	Start gate triggered
3	Started	2 Robot at start	Start gate triggered	Send < TimerState,3 > to RATS Send < C1SplitTime,0 > to RATS Wait for start gate to be cleared	4 Running	Start gate cleared
4	Running	3 Started	Start gate cleared	Send < TimerState,4 > to RATS Wait for finish gate triggered or robot present in start to be triggered.	5 Finishing 2 Robot at Start	Finish gate triggered Robot at start triggered
5	Finishing	4 Running	Finish gate triggered	Send < TimerState,5 > to RATS Send < C1RunTime,value > to RATS Send < C1RunTime,value > to RATS Send < C1RunTime,value > to RATS Wait for finish gate to be cleared	1 Waiting Robot	Finish gate cleared
9	New Entry	0 Calibrating 1 Waiting Robot 2 Robot at start 3 Started 4 Running 5 Finishing	NewMouse	Initialise gates and gate timer Receive ContestName Receive EventName Receive AllowedRuns Receive EntryTimeS Send < CourseTimeMs,0 > to RATS	1 Waiting Robot	Immediate exit after actions

Preferred message sequences for communicating with RATS (Registration And Timing System)

7 Example 2 Gate Line Follower State machine for timing actions

7.1 2 Gate State Diagram



7.2 2 Gate State Actions

State	Name	Predecessors	Entry condition	Actions	Successors	Exit Condition
-------	------	--------------	-----------------	---------	------------	----------------

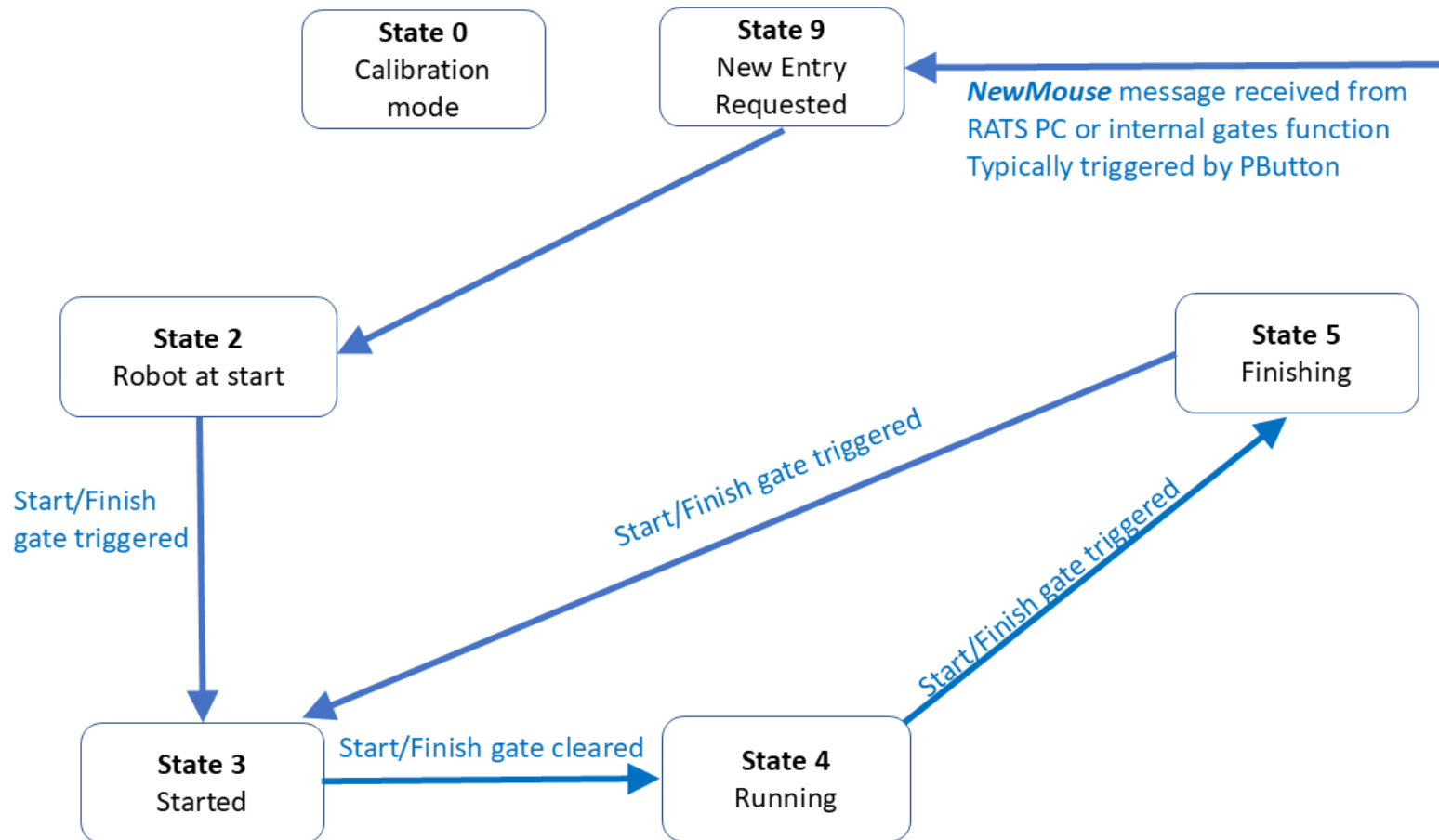
Preferred message sequences for communicating with RATS (Registration And Timing System)

0	Calibrating	Initial state	Power On	Wait NewMouse from RATS	9 New Entry	NewMouse received
1	Waiting Robot			State not used		
2	Robot at Start	9 New Entry 5 Finishing	Immediate Finish gate cleared	Send < TimerState,2 > to RATS Wait for start gate to be triggered	3 Started	Start gate triggered
3	Started	2 Robot at start 4 Running	Start gate triggered	Send < TimerState,3 > to RATS Send < C1SplitTime,0 > to RATS Wait for start gate to be cleared	4 Running	Start gate cleared
4	Running	3 Started	Start gate cleared	Send < TimerState,4 > to RATS Wait for finish gate triggered or start gate triggered.	5 Finishing 3 Started	Finish gate triggered Start gate triggered
5	Finishing	4 Running	Finish gate triggered	Send < TimerState,5 > to RATS Send < C1RunTime,value > to RATS Send < C1RunTime,value > to RATS Send < C1RunTime,value > to RATS Wait for finish gate to be cleared	2 Robot at Start	Finish gate cleared
9	New Entry	0 Calibrating 2 Robot at start 3 Started 4 Running 5 Finishing	NewMouse	Initialise gates and gate timer Receive ContestName Receive EventName Receive AllowedRuns Receive EntryTimeS Send < CourseTimeMs,0 > to RATS	2 Robot at Start	Immediate exit after actions

Preferred message sequences for communicating with RATS (Registration And Timing System)

8 Example 1 Gate Line Follower State machine for timing actions

8.1 1 Gate State Diagram



8.2 1 Gate State Actions

State	Name	Predecessors	Entry condition	Actions	Successors	Exit Condition
-------	------	--------------	-----------------	---------	------------	----------------

Preferred message sequences for communicating with RATS (Registration And Timing System)

0	Calibrating	Initial state	Power On	Wait NewMouse from RATS	9 New Entry	NewMouse received
1	Waiting Robot			State not used		
2	Robot at Start	9 New Entry	Immediate	Send < TimerState,2 > to RATS Wait for start gate to be triggered	3 Started	Start/Finish gate triggered
3	Started	2 Robot at start	Start/Finish gate triggered	Send < TimerState,3 > to RATS Send < C1SplitTime,0 > to RATS Wait for start gate to be cleared	4 Running	Start/Finish gate cleared
4	Running	3 Started	Start/Finish gate cleared	Send < TimerState,4 > to RATS Wait for finish gate triggered or start gate triggered.	5 Finishing	Start/Finish gate triggered
5	Finishing	4 Running	Finish gate triggered	Send < TimerState,5 > to RATS Send < C1RunTime,value > to RATS Send < C1RunTime,value > to RATS Send < C1RunTime,value > to RATS	2 Robot at Start	Immediate exit after actions
9	New Entry	0 Calibrating 2 Robot at start 3 Started 4 Running 5 Finishing	NewMouse	Initialise gates and gate timer Receive ContestName Receive EventName Receive AllowedRuns Receive EntryTimeS Send < CourseTimeMs,0 > to RATS	2 Robot at Start	Immediate exit after actions

Preferred message sequences for communicating with RATS (Registration And Timing System)

9 Annex A- All RATS message types

ENCODED	MESSAGE TYPE	DIRECTION	TX FREQUENCY	COMMENTS
1	CourseTime	Arduino to PC	1T or 100msec	Length of time in 100 millisecond units that the current mouse has been active in the maze
2	SplitTime	Arduino to PC	1T or 10 msec	Legacy compatibility: Length of time in 10 millisecond units for the current mouse on its current run (only used to display current run time)
3	RunTime	Arduino to PC	Event Driven	Legacy compatibility: Length of time in 10 millisecond units for a run that has just completed (definitive time used to calculate score time)
4	TimerState	Arduino to PC	Event Driven	Timer state reported from arduino (Valid Values: any integer)
5	RunTimeMs	Arduino to PC	Event Driven	Length of time in 1 millisecond units for a run that has just completed (definitive time used to calculate score time)
6	SplitToRun	Arduino to PC	Event Driven	Copy the split time to the run time and store runtime so RATS does all the timing based on messages received. (value argument must be 0 or no action will be taken)
71	STrigger	Arduino to PC	Event Driven	Legacy compatibility: New value of Start Gate trigger (Valid values: ON, OFF)
72	FTrigger	Arduino to PC	Event Driven	Legacy compatibility: New value of Finish Gate trigger (Valid values: ON, OFF)
73	CTrigger	Arduino to PC	Event Driven	Legacy compatibility: New value of Mouse in Start Cell trigger (Valid values: ON, OFF)
81	SGLevel	Arduino to PC	100 msec	Legacy compatibility: Intensity level being received by Start Gate phototransistor
82	SGPot	Arduino to PC	100 msec	Legacy compatibility: Value read from Start Gate potentiometer
83	FGLevel	Arduino to PC	100 msec	Legacy compatibility: Intensity level being received by Finish Gate phototransistor
84	FGPot	Arduino to PC	100 msec	Legacy compatibility: Value read from Finish Gate potentiometer
85	SCLevel	Arduino to PC	100 msec	Legacy compatibility: Intensity level being received by Mouse in Start Cell phototransistor
86	SCPot	Arduino to PC	100 msec	Legacy compatibility: Value read from Mouse in Start Cell potentiometer
91	RequestTouch	Arduino to PC	Event Driven	Gate timer (aka Arduino) requests RATS to add a touch to the current run (Value will always be 1)
92	ExtraRun	PC to Arduino	Event Driven	RATS requests the Gate timer (aka Arduino) to allow an extra run by decrementing the run count (Value will always be 1)
93	EntryTimeS	PC to Arduino	Event Driven	Sent following a NewMouse message (value argument is the time in seconds allowed for this entry including grace time)
94	AllowedRuns	PC to Arduino	Event Driven	Sent following a NewMouse message (value argument is the number of runs allowed for this entry)
95	EventName	PC to Arduino	Event Driven	

Preferred message sequences for communicating with RATS (Registration And Timing System)

96	ContestName	PC to Arduino	Event Driven	Sent following a NewMouse message (value argument is the name of the robotics event that is in progress (no quotation marks). eg Minos 2026 Sent following a NewMouse message (value argument is the name of the contest that is in progress (no quotation marks) eg Senior Maze Solver
98	NewMouse	PC to Arduino	Event Driven	A new mouse has been selected in the windows supervision application (value argument is the name of the mouse (no quotation marks))
99	SetMode	PC to Arduino	Event Driven	Controls the Arduino mode (Valid values: TIMER (normal timing mode), CALIBRATION (start returning calibration data))
97	RequestType	PC to Arduino	Event Driven	Request from PC for timer to identify its type (value argument will always be passed as 0)
96	TimerType	Arduino to PC	Event Driven	Type of timer (Valid Values: 1CH (1 channel timer), 2CH (2 channel timer))
11	C1Reaction	Arduino to PC	Event Driven	Channel 1: Length of time in milliseconds between run start and start gate trigger (must be sent before C1RunTime)
12	C1SplitTime	Arduino to PC	10 msec	Channel 1: Length of time in milliseconds for the current mouse on its current run (only used to display current run time)
13	C1RunTime	Arduino to PC	Event Driven	Channel 1: Length of time in milliseconds for a run that has just completed (definitive time used to calculate score time)
14	C1STrigger	Arduino to PC	Event Driven	Channel 1: New value of Start Gate trigger (Valid values: ON, OFF)
15	C1FTtrigger	Arduino to PC	Event Driven	Channel 1: New value of Finish Gate trigger (Valid values: ON, OFF)
16	C1CTrigger	Arduino to PC	Event Driven	Channel 1: New value of Mouse in Start Cell trigger (Valid values: ON, OFF)
17	C1SGLevel	Arduino to PC	100 msec	Channel 1: Intensity level being received by Start Gate phototransistor
18	C1FGLevel	Arduino to PC	100 msec	Channel 1: Intensity level being received by Finish Gate phototransistor
19	C1SCLevel	Arduino to PC	100 msec	Channel 1: Intensity level being received by Mouse in Start Cell phototransistor
21	C2Reaction	Arduino to PC	Event Driven	Channel 2: Length of time in milliseconds between run start and start gate trigger (must be sent before C2RunTime)
22	C2SplitTime	Arduino to PC	10 msec	Channel 2: Length of time in milliseconds for the current mouse on its current run (only used to display current run time)

Preferred message sequences for communicating with RATS (Registration And Timing System)

23	C2RunTime	Arduino to PC	Event Driven	Channel 2: Length of time in milliseconds for a run that has just completed (definitive time used to calculate score time)
24	C2STrigger	Arduino to PC	Event Driven	Channel 2: New value of Start Gate trigger (Valid values: ON, OFF)
25	C2FTTrigger	Arduino to PC	Event Driven	Channel 2: New value of Finish Gate trigger (Valid values: ON, OFF)
26	C2CTrigger	Arduino to PC	Event Driven	Channel 2: New value of Mouse in Start Cell trigger (Valid values: ON, OFF)
27	C2SGLevel	Arduino to PC	100 msec	Channel 2: Intensity level being received by Start Gate phototransistor
28	C2FGLevel	Arduino to PC	100 msec	Channel 2: Intensity level being received by Finish Gate phototransistor
29	C2SCLLevel	Arduino to PC	100 msec	Channel 2: Intensity level being received by Mouse in Start Cell phototransistor
30	CourseTimeM	Arduino to PC	Event Driven	Alternative higher resolution to message type 1. Length of time in milliseconds that the current mouse has been active in the maze